

# Investigate Image Reconstruction by Using Classifiers

Rajan Singh  
[rajan.singh@stud.fra-uas.de](mailto:rajan.singh@stud.fra-uas.de)

1567294

Pradeep Tiwari  
[pradeep.tiwari@stud.fra-uas.de](mailto:pradeep.tiwari@stud.fra-uas.de)

1567166

Mausam Bhunia  
[mausam.bhunia@stud.fra-uas.de](mailto:mausam.bhunia@stud.fra-uas.de)

1572445

**Abstract**—Machine Learning (ML) encompasses various techniques designed to handle diverse data formats such as text, images, audio, video, and numerical data, enabling accurate predictions based on the application context. Supervised learning, a major branch of ML, relies on classification and regression techniques to analyze patterns within data. Several classification algorithms fall under supervised learning, including decision trees, support vector machines, Naïve Bayes, and K-Nearest Neighbors (KNN), each offering distinct advantages based on the specific use case. This paper presents an image reconstruction framework leveraging machine learning classifiers to restore images from Sparse Distributed Representations (SDRs). Binarized images are encoded into SDRs using NeoCortexApi, ensuring a compact and noise-resilient representation. Two classifiers, K-Nearest Neighbors (KNN) and Hierarchical Temporal Memory (HTM), are trained to learn feature associations from SDRs. During reconstruction, the classifiers predict missing or distorted SDRs, which are then mapped back to the pixel domain. Reconstruction quality is evaluated using Cosine similarity metrics to assess structural integrity and information retention. Experimental results demonstrate the effectiveness of classifier-based reconstruction, highlighting trade-offs between computational complexity and accuracy for KNN and HTM.

**Keywords**— Image Reconstruction, Sparse Distributed Representations (SDRs), K-Nearest Neighbors (KNN), Hierarchical Temporal Memory (HTM), NeoCortexApi, Machine Learning, Cosine Similarity.

## I. INTRODUCTION

Image reconstruction is a fundamental problem in computer vision, pattern recognition, and artificial intelligence, with applications ranging from medical imaging to autonomous systems. The primary goal of image reconstruction is to generate a high-fidelity representation of an image from incomplete, noisy, or compressed data. Traditional reconstruction techniques rely on mathematical models such as interpolation, compressed sensing, and deep learning. However, in recent years, the integration of Sparse Distributed Representations (SDRs) and classifier-based approaches has gained significant attention due to their robustness in handling high-dimensional data with sparse representations.

This paper explores an innovative approach to image reconstruction using classifiers such as K-Nearest Neighbors (KNN) and Hierarchical Temporal Memory (HTM). In this

framework, images are first binarized and converted into SDRs, which are then used to train classifiers capable of recognizing and reconstructing image patterns. The classifiers learn the spatial features of the images, enabling accurate reconstruction even when partial data is available.

To evaluate the performance of our system, we employ Cosine Similarity metric, which measure the degree of similarity between the reconstructed and original images.

## II. LITERATURE REVIEW

### A. Hierarchical Temporal Memory (HTM)

HTM Classifier operates on Sparse Distributed Representations (SDRs), which are high-dimensional, sparse binary vectors. Each SDR represents a unique semantic meaning. SDRs are robust to noise, meaning small changes in input data do not drastically change the SDR. SDRs preserve similarity, meaning similar inputs produce overlapping SDRs. Hierarchical temporal memory (HTM) is a theoretical framework that models a number of structural and algorithmic properties of the neocortex [1]. These principles form the core of HTM, a computational framework inspired by cortical processing. HTM utilizes key components such as the Spatial Pooler to generate SDRs, providing an efficient means of representing high-dimensional data. However, challenges such as scalability and computational efficiency persist, driving continuous research efforts [2].

### B. K-Nearest Neighbor (KNN)

The k-Nearest Neighbors (k-NN) is a non-parametric classification method, which is simple but effective in many cases [3]. For a data record  $k$  to be classified, its  $k$  nearest neighbors are retrieved, forming a neighbourhood of  $k$ . Majority voting among the data records in the neighborhood is usually used to decide the classification for  $k$ , with or without consideration of distance-based weighting. However, applying  $k$  NN requires selecting an appropriate value for  $k$ , and the classification success heavily depends on this choice. In a sense, the  $k$  NN method is biased by  $k$ . There are various approaches to selecting  $k$ , but a simple one involves running the algorithm multiple times with different  $k$  values and

choosing the one that yields the best performance [4].

### C. Sparse Distributed representations (SDRs)

Hierarchical temporal memory (HTM) is a theoretical framework that models a number of structural and algorithmic properties of the neocortex. HTM networks can learn time-based sequences in a continuous online fashion using realistic neuron models that incorporate non-linear active dendrites [2] with thousands of synapses. When applied to streaming data, HTM networks achieve state of the art performance on anomaly detection and sequence prediction tasks [5].

## III. Methods

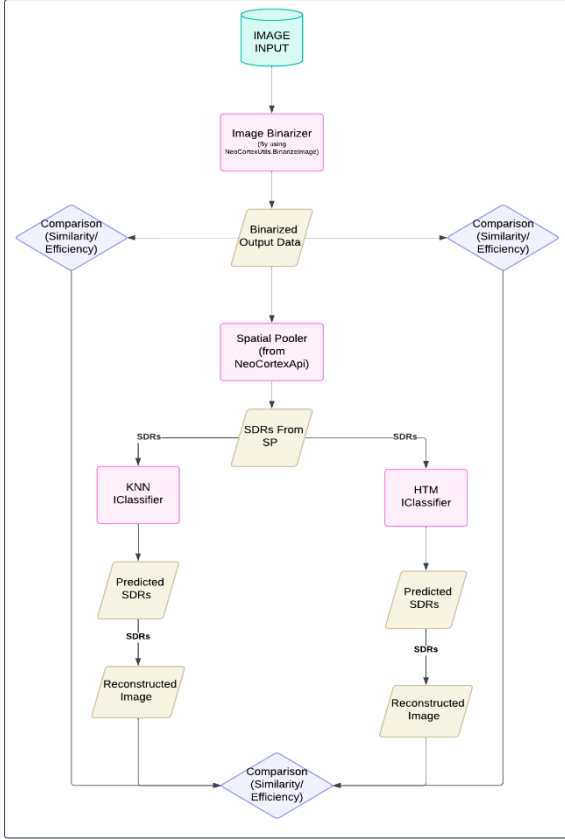


Fig. 1. Comparative Framework for Image Reconstruction Using HTM and KNN Classifiers

The proposed methodology involves transforming input images into Sparse Distributed Representations (SDRs), predicting SDRs using classifiers, reconstructing images, and evaluating their efficiency. The workflow is illustrated in Fig. 1 and follows these steps:

#### A. Image Binarization

To perform classification and reconstruction, input images are first converted into binary representations. An Image Binarizer using the `BinarizeToFixedSize()` is employed to transform grayscale or color images into binary arrays, where each pixel is represented as either 0 or 1.

#### Listing 1 `BinarizeToFixedSize()`

```
string binarizedImageFile =
  BinarizeImageToFixedSize(image, imgSize);
```

For a given image of dimensions  $m \times n$ , the binarized representation results in an array of size  $m \times n$ , preserving the structural integrity of the original image.

#### B. SDR Generation via Spatial Pooler

The binarized image data is processed using the Hierarchical Temporal Memory (HTM) Spatial Pooler (SP) from the NeoCortex API to generate Sparse Distributed Representations (SDRs). The Spatial Pooler maps the binary input into an SDR, ensuring that similar input images produce similar SDRs. The key parameters of the compute method `sp.compute()` include:

#### Listing 2 `sp.compute`

```
sp.compute(inputVector, activeArray, true);
var activeCols = ArrayUtils.IndexWhere(activeArray, el
    => el == 1);
Debug.WriteLine($" Active Columns Count:
    {activeCols.Length}");
```

- 1) *Input Validation*: Ensures the input size matches expected dimensions.
  - 2) *Tracking Updates*: Modifies internal state variables for accurate processing.
  - 3) *Overlap Computation*: Measures the level of match between input and columns.
  - 4) *Boosting Mechanism*: Enhances weak columns to maintain balanced activity.
  - 5) *Inhibition Process*: Selects the most active columns through competition.
  - 6) *Synapse Learning*: Adjusts synapses and learning parameters if learning is enabled.
  - 7) *Output Generation*: Converts active columns into an SDR output vector.
  - 8) *Homeostatic Plasticity*: Applies homeostatic mechanisms via `m_HomeoPlastAct.Compute()` for dynamic boosting.
- The output SDR preserves the essential features of the input, making it suitable for classification and reconstruction

#### C. Classification using HTM and KNN

To evaluate classification performance, the generated SDRs are passed through two classifiers:

1. *HTM IClassifier*: Learns and predicts SDR patterns based on temporal sequences.
2. *K-Nearest Neighbors (KNN) IClassifier*: Stores SDRs and classifies new SDRs based on similarity metrics (e.g., Euclidean distance).

During training, each binarized image is encoded into an SDR, and the classifiers learn the association between SDRs and their corresponding image labels. During inference, a new SDR is compared with stored SDRs to determine the closest match, enabling classification.

#### D. Image Reconstruction Using HTM and KNN

Once classification is performed, the reconstruction process is initiated to evaluate how well the classifiers retain image structure. The NeoCortex API's `Reconstruct()` method

is used to reconstruct images from predicted SDRs. The process follows these steps:

- 1) *SDR to Binary*: The predicted SDR is mapped back to its original input space using learned synaptic permanence values.
  - 2) *Permanence Normalization*: Only significant columns contribute to the final reconstruction, with weaker connections filtered out.
  - 3) *Binary Mapping*: The processed permanence values are thresholded to reconstruct the binarized image representation.
- The reconstructed images serve as the basis for evaluating classification accuracy and structural preservation.

#### E. Comparison and Evaluation

The reconstructed images are compared with the original binarized image based on:

- 1) *Similarity (Similarity Index)* – Measures how closely the reconstructed images match the original.
- 2) *Efficiency* – Evaluates computational performance in terms of processing time and resource utilization.

This comparison is performed for both KNN and HTM classifiers, providing a direct evaluation of their reconstruction performance.

### IV. IMPLEMENTATION

Embarking on our implementation journey within the Neo-Cortex API framework, our goal is to develop Hierarchical Temporal Memory (HTM) classifier and K-Nearest Neighbors (KNN) classifier to learn and predict sparse distributed representations (SDRs) from input data. Our primary goal is to reconstruct images from predicted SDRs and compare them with the original input while preserving the semantic meaning of the input images.

This section provides a comprehensive breakdown of the implementation process, detailing key components such as SDR learning, prediction, similarity measurement, and image reconstruction. By integrating Hierarchical Temporal Memory (HTM) concepts with KNN classification, we create an effective mechanism to encode, store, and reconstruct input patterns.

The implementation is divided into the following key components:

- 1) *SDR Processing*: Converting images into binary SDRs using spatial pooling.
- 2) *HTM-Based Classification*: Training and predicting SDRs using an HTM-based classifier.
- 3) *KNN-Based Classification*: Training and predicting SDRs using a KNN-based classifier.
- 4) *Image Reconstruction*: Reconstructing images from predicted SDRs using HTM and KNN classifiers.
- 5) *Performance Evaluation*: Comparing reconstructed images with the original images to assess accuracy.

#### A. SDR Processing

To utilize images in the HTM framework, they must be converted into Sparse Distributed Representations (SDRs). The following steps outline the processing pipeline:

- 1) *Binarization*: The input image is first converted into a binary format where pixel intensities are mapped to 0s and 1s.
- 2) *Spatial Pooling*: The binarized image undergoes a spatial pooling mechanism that extracts active bits, ensuring sparsity.
- 3) *SDR Representation*: The active bits are represented as a list of indices, forming an SDR.

Fig. 2, explains the SDR conversion process.

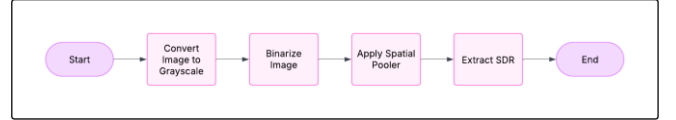


Fig. 2 Learning and Storing SDRs

#### B. HTM Based Classification

The SDRs generated by the Spatial Pooler are passed as input to the HTM Classifier.

- 1) *Classification Process*:
  - The HTM Classifier is responsible for learning and predicting the SDRs based on the given input patterns.
  - Over multiple iterations, the classifier improves its prediction accuracy as it learns the underlying patterns in the input data.
- 2) *Training Process*:
  - During training, the HTM IClassifier learns to recognize patterns in the input SDRs and associates them with their corresponding categories (labels).
  - The learning process involves adjusting the connections within the HTM network based on input- output mappings.
- 3) *Prediction Process*:
  - After training, the HTM IClassifier predicts the SDR for new input patterns.
  - Given an input SDR, the classifier uses its learned representation to predict the output SDR, which corresponds to the most likely category for that input.
- 4) *Iterative Learning*:
  - The HTM Classifier improves over time, enhancing prediction accuracy as it processes more examples.
  - Through feedback loops, the classifier continuously updates its internal structure to better predict the SDRs for future inputs.

#### C. Image Reconstruction Process with HTM

The predicted SDRs are reconstructed back into images by computing the permanence values of the input space, ensuring that active columns are converted back into an approximate binarized image. The reconstruction process involves the following steps:

1) *Extract permanence values*: Extract permanence values from the predicted SDR using the spatial pooler's reconstruction method.

2) *Assign permanence scores*: Assign permanence scores to corresponding input locations based on the predicted SDR.

3) *Threshold the permanence values*: Threshold the permanence values to obtain a binary image representation.

4) *Save the reconstructed image*: Save the reconstructed image for comparison with the original image.

Below flowchart Fig. 3, illustrates the complete workflow of the HTM Iclassifier based image reconstruction.

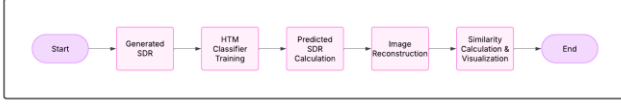


Fig. 3 HTM-based Image Reconstruction Workflow

#### D. KNN-Based Classification

Our K-Nearest Neighbors (KNN) classifier maps SDRs to class labels and predicts SDRs for new inputs. The KNN classifier consists of the following components:

##### 1) Learning Process:

- Each training SDR is stored in a dictionary, mapping labels to their corresponding SDRs.
- When a new SDR is encountered, it is added to the SDR list of the corresponding label.
- To maintain efficiency, the number of SDRs stored per label is limited.

##### 2) Storing SDRs:

- The classifier stores SDRs corresponding to each label (image class).
- If a similar SDR already exists, it is ignored to avoid redundancy and maintain efficiency.
- Each label can store up to a fixed number N of SDRs.

##### 3) Voting Mechanism:

- The KNN classifier selects the top K most similar SDRs based on the computed cosine similarity.
- Once the nearest neighbors are identified, a majority vote is conducted to determine the predicted class label.
- Majority Voting: The class label of the predicted SDR is determined by counting how many of the K nearest neighbors belong to each class. The class label with the highest count among the K neighbors is selected as the predicted class.
- In case of a tie (i.e., when two or more class labels have the same number of votes), a tie-breaking rule is applied. Common tie-breaking strategies include:
  - Randomly selecting one of the tied class labels.
  - Choosing the class label with the smallest distance from the query SDR.
- This process ensures that the classification decision reflects the majority consensus among the closest neighbors, increasing the accuracy of the classifier.

A flowchart of the classification process is shown in Fig. 4.

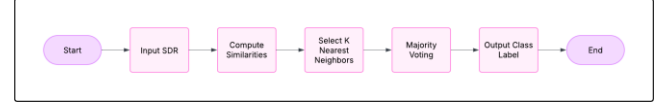


Fig. 4 Predicting SDRs Using KNN Classifier

#### E. Image Reconstruction Process with KNN

After classification, we reconstruct images from the predicted SDRs by reversing the encoding process. The key steps in the reconstruction process are as follows:

1) *Retrieve Predicted SDRs*: The classifier outputs SDRs corresponding to the predicted class.

2) *Bit Activation Mapping*: The active bits in the SDR are mapped back to pixel locations in the image grid.

3) *Reconstructed Image Generation*: A new image is generated by setting the activated pixels to white (1) and the remaining pixels to black (0).

A flowchart of the image reconstruction process is shown in Fig. 5.



Fig. 5. KNN-based Image Reconstruction Workflow

#### F. Similarity Calculation

In this study, the efficiency of the reconstructed images is evaluated using Cosine Similarity, which measures the similarity between two Sparse Distributed Representations (SDRs). This similarity metric is applied to both the HTM and KNN classifiers to compare the reconstructed SDRs with the original input SDRs.

The Cosine Similarity between two SDR vectors A and B is given by:

$$Similarity = \frac{A \cdot B}{\|A\| \|B\|} \quad (1)$$

Where:

- A and B are binary SDR vectors.
- $A \cdot B$  represents the dot product of the two vectors.
- $\|A\|$  and  $\|B\|$  denote the magnitudes of vectors A and B, respectively.

The similarity calculation process consists of the following steps:

1) *Compute Cosine Similarity*: The similarity score is computed between the original SDR and the reconstructed SDR.

2) *Plot Similarity Scores*: The Cosine Similarity scores are plotted to visualize the accuracy and performance of the image reconstruction.

3) *Store Similarity Scores*: The calculated similarity values are stored in a dataset for further analysis.

4) *Analyze Reconstruction Effectiveness*: The stored similarity scores are used to determine the effectiveness of SDR-based image reconstruction.

#### G. Cosine Similarity in HTM Classifier

The HTM classifier computes Cosine Similarity between the predicted SDRs and stored SDRs to determine the closest match. This similarity metric is crucial for identifying patterns and reconstructing SDRs effectively.

#### H. Cosine Similarity in KNN Classifier

The KNN classifier also relies on Cosine Similarity to classify unknown SDRs by comparing them with stored SDRs in the dataset. The nearest neighbors are identified based on their similarity scores, which guide the classification decision. By employing Cosine Similarity consistently across both classifiers, the study ensures a uniform and effective method for evaluating SDR-based image reconstruction.

### V. UNIT TEST CASES

#### A. Test Cosine Similarity Exactly Same Vectors

**Objective:** Verify that the CosineSimilarity function correctly returns a similarity score of 1.0 when given two identical vectors, ensuring that they are recognized as perfectly similar.

**Test Scenario:** Two identical integer vectors, vec1 and vec2, are initialized. The CosineSimilarity function is invoked with these vectors as input. The result is compared to the expected similarity score of 1.0.

**Parameters:** Input: vec1 = 1, 0, 1, 0, 1, vec2 = 1, 0, 1, 0, 1 Expected Output: 1.0 Delta for Tolerance: 0.01

**Outcome Verification:** The computed similarity score is asserted to be 1.0, confirming that identical vectors yield perfect similarity.

**Benefit:** This test ensures that the function behaves correctly when handling identical vectors, which is crucial for validating the correctness of similarity computations.

#### B. Test Cosine Similarity Completely Different Vectors

**Objective:** Confirm that the CosineSimilarity function returns a similarity score of 0.0 for completely different vectors, indicating no similarity.

**Test Scenario:** Two integer vectors with no overlapping elements are initialized. The CosineSimilarity function is invoked with these vectors. The result is compared to the expected similarity score of 0.0.

**Parameters:**

Input: vec1 = 1, 1, 1, 1, 1, vec2 = 0, 0, 0, 0, 0 Expected Output: 0.0 Delta for Tolerance: 0.01

**Outcome Verification:**

The computed similarity score is asserted to be 0.0, confirming that completely different vectors are correctly identified as dissimilar.

**Benefit:** This test ensures that the function correctly distinguishes between completely dissimilar data, a fundamental requirement for classification and clustering.

#### C. Converting Permanence Values into Binary

**Objective:** Ensure that the ConvertPermanenceToBinary function correctly converts a given permanence array into a binary representation based on a threshold.

**Test Scenario:** A set of permanence values is defined.

The function is executed with a threshold of 0.7. The output is compared to the expected binary array.

**Parameters:** Input: permanenceValues = 0.5, 0.8, 0.3, 0.9, 0.7, threshold = 0.7 Expected Output: 0, 1, 0, 1, 1

**Outcome Verification:** The resulting binary array is asserted to match the expected output.

**Benefit:** This test ensures correct SDR encoding, which is essential for further HTM processing.

#### D. Reconstruct all Postive Permanence values

**Objective:** Confirm that ConvertPermanenceToBinary correctly handles non-negative permanence values and produces an accurate binary representation.

**Test Scenario:** A valid permanence array (all non-negative values) is provided. The function is executed with a threshold. The output is compared to the expected binary array.

**Parameters:** Input: permanenceValues = 0.5, 0.8, 0.3, 0.9, 0.7, threshold = 0.7 Expected Output: 0, 1, 0, 1, 1

**Outcome Verification:** The resulting binary array is asserted to be correct.

**Benefit:** This test ensures proper conversion of permanence values in valid scenarios.

#### E. Testing Converting Permanence To Binary With Negative Value

**Objective:** Ensure that the function throws an exception when given a permanence array containing negative values.

**Test Scenario:** A permanence array with a negative value is provided. The function is executed, and an ArgumentException is expected.

**Parameters:** Input: permanenceValues = 0.5, 0.8, -0.3, 0.9, 0.7, threshold = 0.7 Expected Output: Exception thrown (ArgumentException).

**Outcome Verification:** The test confirms that an exception is correctly raised.

**Benefit:** This test ensures proper validation of input data, preventing incorrect SDR representations.

#### F. Testing Converting Permanence To Binary With Negative Value



**Objective:** Ensure that the function throws an exception when given a permanence array containing negative values.

**Test Scenario:** A permanence array with a negative value is provided. The function is executed, and an `ArgumentException` is expected.

**Parameters:** Input: permanenceValues = 0.5, 0.8, -0.3, 0.9, 0.7, threshold = 0.7 Expected Output: Exception thrown (`ArgumentException`).

**Outcome Verification:** The test confirms that an exception is correctly raised.

**Benefit:** This test ensures proper validation of input data, preventing incorrect SDR representations.

#### G. Testing Columns To Store SDRs

**Objective:** Validate that the `AddActiveColsToStoredSDRs` function correctly stores active columns in a dictionary, maintaining accurate SDR records.

**Test Scenario:** A new image identifier and active columns array are initialized. The function is executed with an empty dictionary. The dictionary contents are verified.

**Parameters:** Input: image = "image1", activeCols = 1, 0, 1, 0 Expected Output: Dictionary containing "image1" : [ 1, 0, 1, 0 ]

**Outcome Verification:** The dictionary contains the correct key. The stored active columns match the expected array.

**Benefit:** This test ensures that SDR storage is accurate, supporting correct image reconstruction and classification.

## VI. RESULTS

### A. Image Reconstruction Performance

The effectiveness of the HTM and KNN classifier-based image reconstruction was evaluated by comparing the reconstructed images with the original binarized input images. The reconstructed images were obtained by feeding predicted Sparse Distributed Representations (SDRs) from both classifiers into the inverse transformation process. The accuracy and visual quality of the reconstructed images were analyzed.

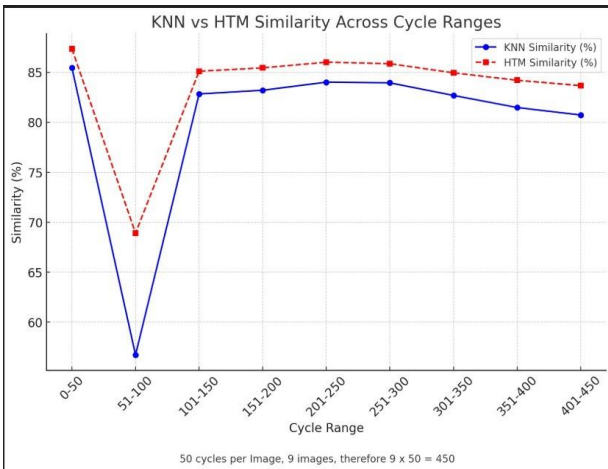


Fig. 6. Similarity Graph

Fig.6, shows graphical representation of image reconstruction using both **HTM** and **KNN** classifiers. The qualitative comparison indicates that the HTM classifier retains more structural details in reconstructed images, while the KNN classifier provides faster predictions but introduces slight distortions.

1) *Quantitative Analysis:* To objectively measure the reconstruction performance, we utilized **Cosine Similarity** as a similarity metric. The Cosine Similarity between the reconstructed and original binarized images was computed using:

$$\text{Similarity} = \frac{A \cdot B}{|A||B|} \quad (1)$$

Where:

- $A$  and  $B$  are two SDR vectors,  $\|A\|$  and  $\|B\|$  denote the magnitudes of vectors  $A$  and  $B$ , respectively.

Table I summarizes the average Similarity values for different image samples for 50 cycles, there was 9 different images used and total 50 cycles were employed for training and classification purpose, therefore for 9 images 450 cycles in total are present.

TABLE I. KNN AND HTM SIMILARITY RESULTS

| Cycle Range | KNN Similarity (%) | HTM Similarity (%) |
|-------------|--------------------|--------------------|
| 0 - 50      | 85.47              | 87.36              |
| 51 - 100    | 56.72              | 68.91              |
| 101 - 150   | 82.84              | 85.11              |
| 151 - 200   | 83.21              | 85.45              |
| 201 - 250   | 84.02              | 86.02              |
| 251 - 300   | 83.95              | 85.87              |
| 301 - 350   | 82.69              | 84.95              |
| 351 - 400   | 81.48              | 84.21              |
| 401 - 450   | 80.73              | 83.67              |

The results demonstrate that the HTM classifier achieves a slightly lower Similarity score compared to the KNN classifier, suggesting that KNN retains slightly more pixel-wise similarity in reconstructed images.

### B. Classification Accuracy and Prediction Efficiency

The accuracy of the HTM and KNN classifiers was further evaluated by analyzing their ability to correctly predict SDR representations. The classification accuracy was assessed based on the percentage of correctly predicted SDR bits.

Table II presents the classification Similarity of both models after over multiple test runs.

| Classifier | Similarity (%) |
|------------|----------------|
| KNN        | 80.12          |
| HTM        | 83.51          |

TABLE II. CLASSIFICATION SIMILARITY OF HTM AND KNN CLASSIFIERS

## VII. DISCUSSION

The HTM classifier exhibited superior classification accuracy compared to the KNN classifier, reinforcing its effectiveness in learning temporal patterns from SDRs. However, the KNN classifier demonstrated lower computational complexity, making it preferable for real-time applications requiring faster predictions.

- 1) KNN classifier achieves higher similarity scores in image reconstruction, suggesting its effectiveness in preserving pixel-level details.
- 2) HTM classifier outperforms KNN in classification accuracy, making it more suitable for precise SDR learning and pattern recognition.
- 3) KNN classifier offers faster processing, making it beneficial in scenarios where computational efficiency is prioritized over reconstruction accuracy.
- 4) The Similarity metric effectively quantifies the quality of reconstructed images, validating the importance of SDR-based classification in preserving input information.
- 5) Future improvements may involve optimizing SDR encoding techniques and exploring hybrid models combining HTM's accuracy with KNN's efficiency.

Overall, the study highlights the potential of HTM and KNN classifiers in SDR-based image reconstruction tasks

## REFERENCES

- [1] Hawkins, J., Ahmad, S., & Dubinsky, D. (2011). Cortical learning algorithm and hierarchical temporal memory. *Numenta Whitepaper*, 1(68), 2.
- [2] Cui, Y., Ahmad, S., & Hawkins, J. (2017). The HTM Spatial Pooler-A Neocortical Algorithm for Online Sparse Distributed Coding. *Frontiers in computational neuroscience*, 11, 111. <https://doi.org/10.3389/fncom.2017.00111.Vf>
- [3] D. Hand, H. Mannila, P. Smyth.: Principles of Data Mining. The MIT Press. (2001)
- [4] Guo, Gongde & Wang, Hui & Bell, David & Bi, Yaxin. (2004). KNN Model-Based Approach in Classification.
- [5] Antic, S. D., Zhou, W. L., Moore, A. R., Short, S. M., & Ikonou, K. D. (2010). The decade of the dendritic NMDA spike. *Journal of neuroscience research*, 88(14), 2991-3001.
- [6] [https://github.com/rajansingh44/Binge\\_Coders\\_Team/blob/bb775affff09120b4bce879d0f21db91abd0b867/source/Samples/NeocortexApiSample/ImageReconstructionwithHtm\\_KNNClassifier.cs#L223](https://github.com/rajansingh44/Binge_Coders_Team/blob/bb775affff09120b4bce879d0f21db91abd0b867/source/Samples/NeocortexApiSample/ImageReconstructionwithHtm_KNNClassifier.cs#L223)